

Digital Electronics Lab  
EE 3082

Lab 5

Jae Sung Hwang (jh5185)  
Vassily Lombard (vtl2110)

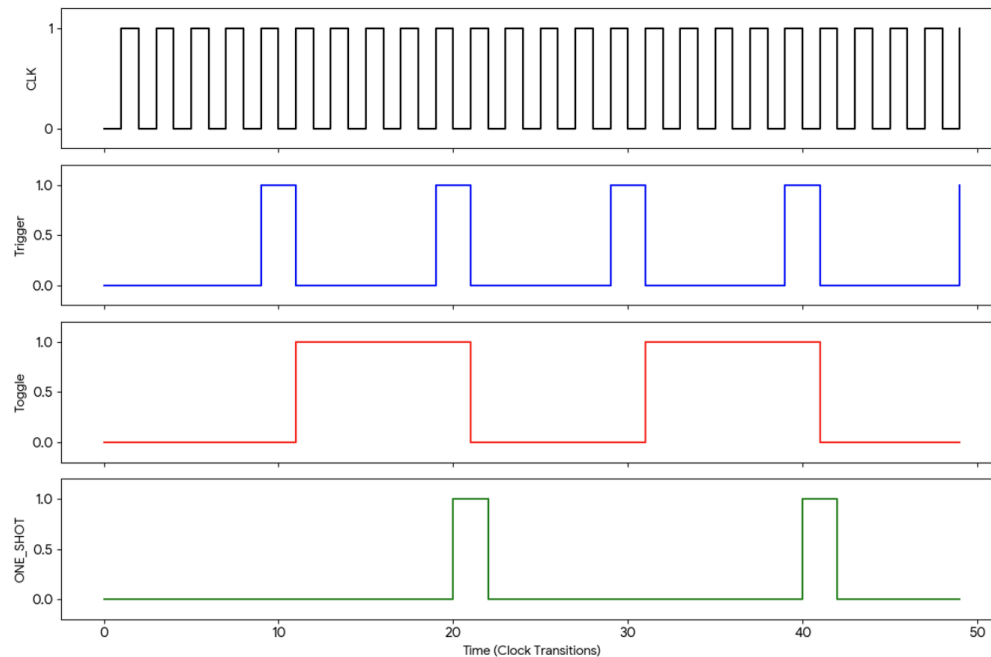
**Question 1:** How are the notes encoded on the switches?

The notes are encoded using a combination of the 8 switches and 4 push-buttons. The 8 switches use one-hot encoding to select the 7 natural notes, meaning exactly one switch must be flipped HIGH at a time. For example, flipping switch 7 (10000000) selects C and maps to `note_sel = 0001`, while flipping switch 3 (00001000) selects G and maps to `1000`. The `note_sel` values are intentionally non-sequential, skipping numbers to leave space for sharps and flats. The push-buttons then modify this selection: PB(1) subtracts 1 from `note_sel` to play a flat, while PB(2) adds 1 to play a sharp so C with PB(2) held gives `C# = 0010`. PB(3) acts as the octave selector by serving as the MSB of the final 5-bit `note_in` signal sent to the note decoder. For example, switch 7 ON with PB(3) held gives `note_in = 1 0001` (decimal 17), representing the upper octave C, while the same switch without PB(3) gives `note_in = 0 0001` (decimal 1), the lower octave C. Any unrecognized switch pattern defaults to `note_sel = 0000`, which effectively produces silence.

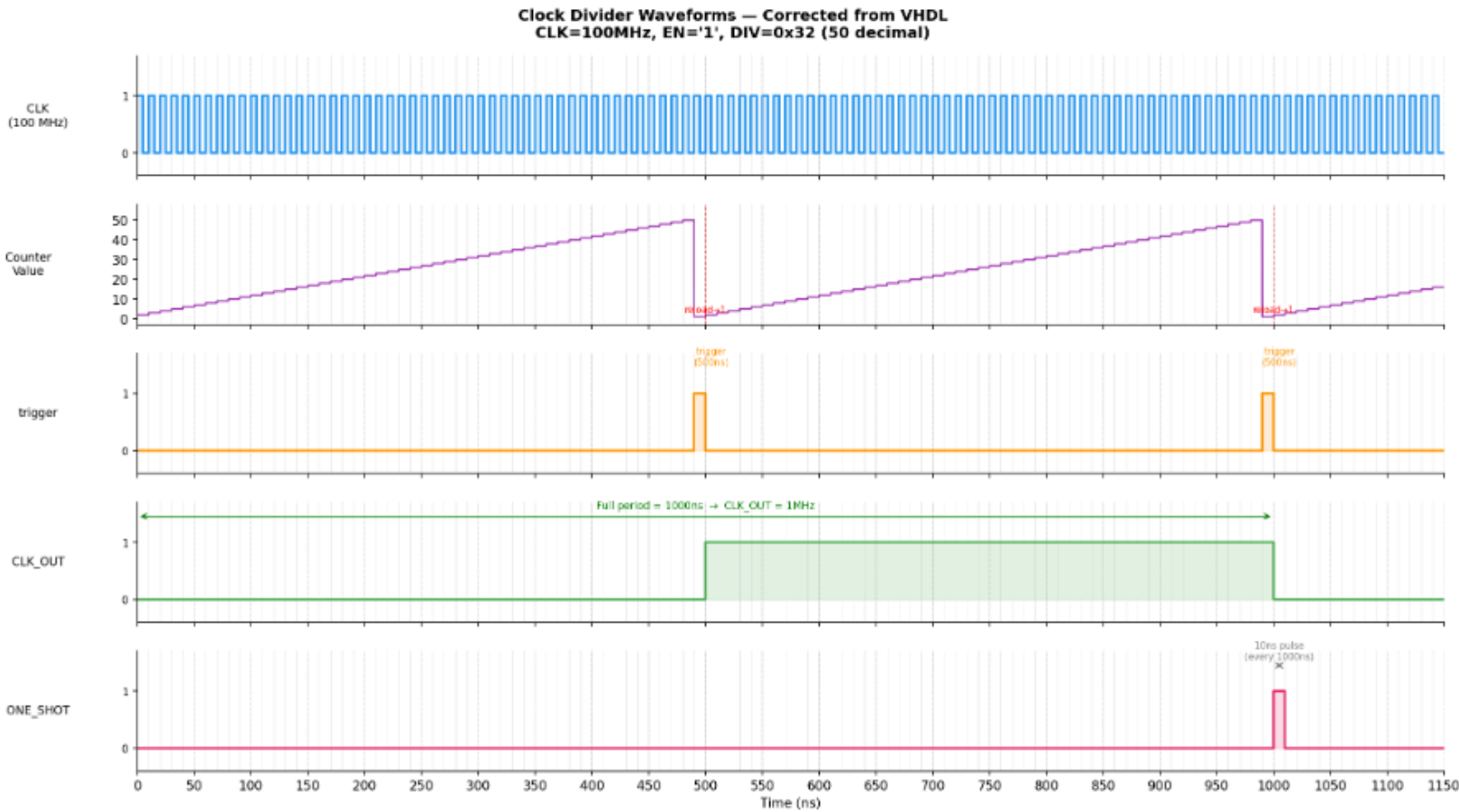
**Question 2:** If CLK is a 100 MHz input clock, sketch the waveforms on CLK\_OUT and ONE\_SHOT for `EN = '1'` and `DIV = X'32'` (32 in hex, which is equal to 50 in decimal representation). You should also sketch the toggle and trigger waveforms; you can trace these signals one cycle at a time to determine their behavior.

The logic of this clock divider centers on a 16-bit counter that acts as the metronome for the rest of the circuit. On every rising edge of the input CLK, the counter increments until it reaches the value of DIV, at which point it resets and fires a single cycle trigger pulse. This trigger acts as a toggle signal for CLK\_OUT, flipping it from high to low (or vice versa) to create a divided square wave with a 50% duty cycle. Because a full clock period requires two of these state flips, the internal toggle register tracks whether we are currently in the first or second half of the cycle; it only allows the ONE\_SHOT pulse to fire when both the trigger and the toggle are high, ensuring a single synchronization pulse occurs exactly once per full output period.

The graph below is an overview of how the different signals work together.



The graph below is a more accurate and descriptive version of the first graph. It shows precisely how the different signals interact between each other, with time and clock counts represented accurately. We can see the original clock signal with the counter, the trigger signal, the output clock and the one shot signal.

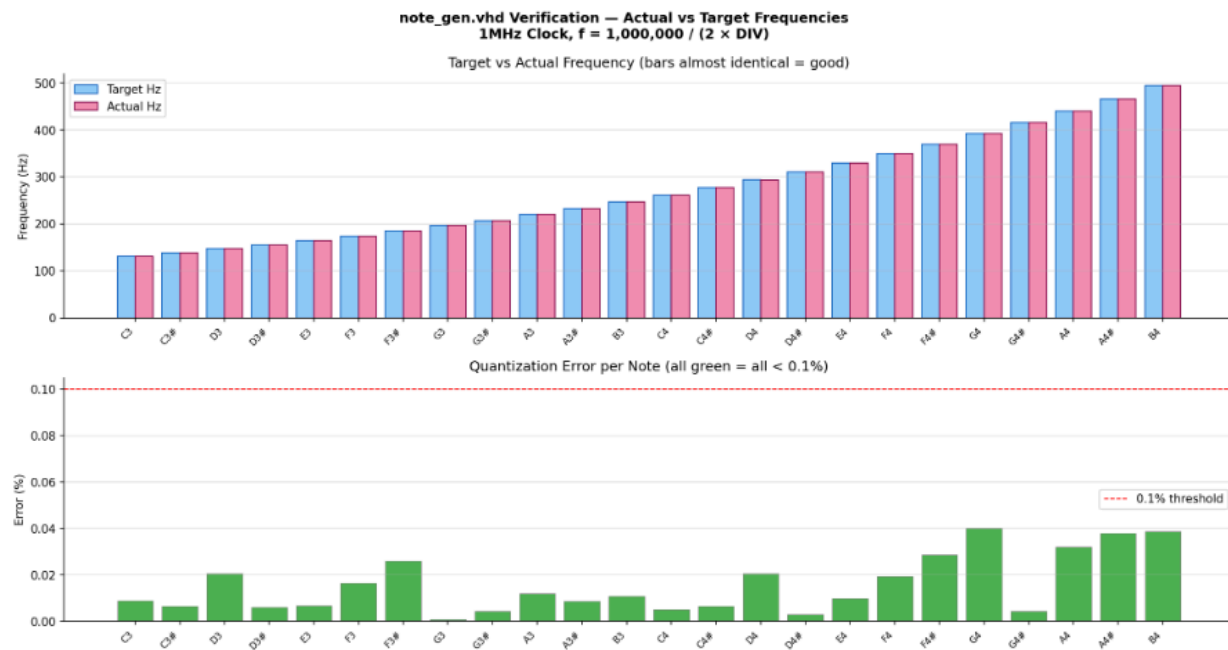


**Question 3: For a 1MHz input, calculate the output frequency as a function of divider value. Combining what you calculate with the table at the top of this document, verify the values used for the note decoder to define the notes of the piano.**

For a 1MHz input clock, the clock divider works by counting down from the loaded DIV value and toggling the output each time it reaches zero, meaning the output period is twice the count interval. This gives an output frequency of  $f_{\text{out}} = 1,000,000 / (2 \times \text{DIV})$ , or equivalently  $\text{DIV} = 500,000 / f_{\text{out}}$ .

To verify a few notes from the frequency table: middle C (C4) at 261.63 Hz requires a divider value of  $500,000 / 261.63 \approx 1911$ , A4 at 440.00 Hz gives  $500,000 / 440 \approx 1136$ , and B4 at 493.88

Hz gives  $500,000 / 493.88 \approx 1013$ . These divider values fall well within the range of a 16-bit counter (max 65,535), confirming that a 1MHz clock with 16-bit granularity provides sufficient resolution across the entire audible piano range from roughly 27.5 Hz (A0,  $DIV \approx 18,182$ ) up to several kHz, consistent with the values used in `note_gen.vhd`.



#### Question 4: Study the implementation in `seven_seg.vhd` and explain how the design functions

The goal of the module is to display the current piano note on the four-digit 7-segment display. First, it takes the 5-bit note input and converts it into a 9-bit buffer that stores three things: the note letter (like C, D, or A) in the upper 4 bits, the octave number (3 or 4) in the lower 4 bits, and a single bit for the decimal point which indicates a sharp note. Then, a 2-bit counter increments on every `SCAN_EN` pulse to cycle through the four digit positions one at a time. On each step, the design activates one digit by pulling its anode signal low while keeping the others high, and loads the appropriate value to display, the octave number on the rightmost digit, the note letter on the second digit, and nothing on the two leftmost digits since only two characters are needed. Finally, a decoder converts each 4 bit value into the correct combination of seven segment signals to physically light up the right shape on the display. Because this scanning cycle runs at around 10kHz, the switching is too fast for the human eye to notice, so all digits appear to be on simultaneously.

So putting it all together the display is constantly cycling through the four digit positions at 10kHz, and on each cycle it shows either the note letter, the octave number, a blank, or another blank. Because this happens so fast, your eye sees all digits lit at once showing something like C4 with a dot if it is a sharp.

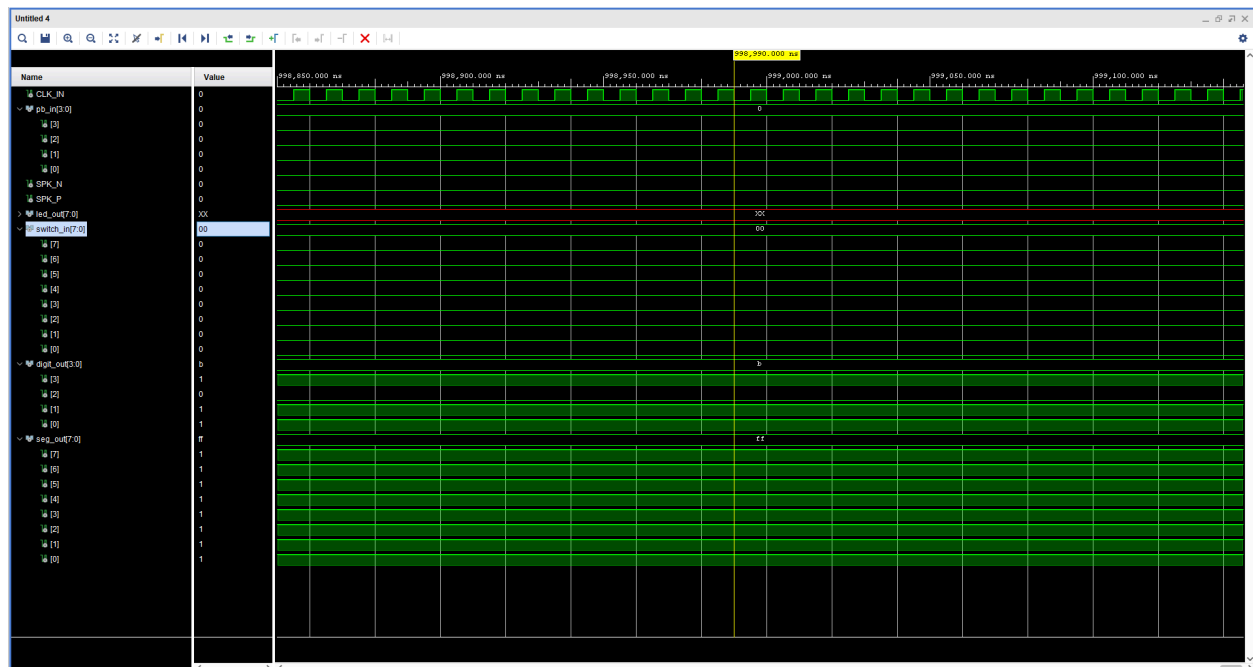
**Question 5: Add additional stimulus to test for various values of pb in and switch in.. Make sure you save the test bench (Ctrl-S)**

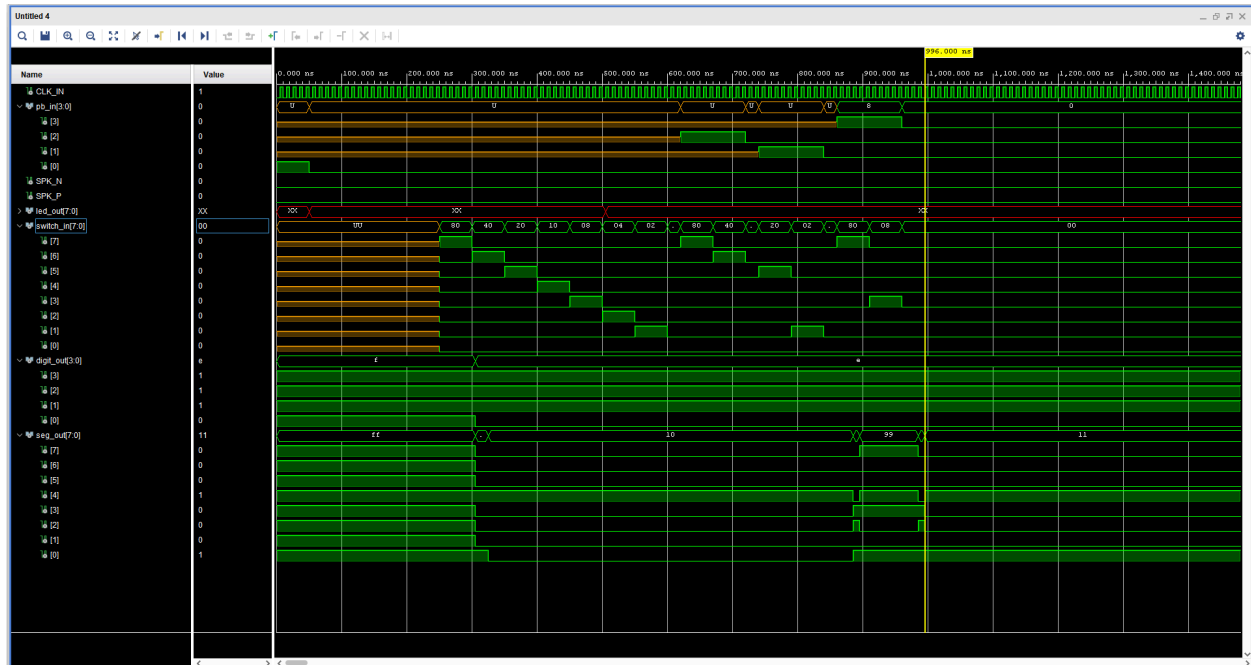
The last two simulations are briefly explained here:

1st one: You can see CLK\_IN toggling, pb\_in showing the reset pulse at the start, and switch\_in cycling through all the note values (80, 40, 20, 10, 08, 04, 02 in hex, those are your notes C through B). That part is all correct.

2nd one: The second simulation stress tests the design by cycling through all seven notes at five progressively faster switching speeds, starting at 200ns per note and going down to near clock speed at 20ns, while also testing random note jumping and rapid toggling between a note and silence.

**Question 6: Save some waveforms from your simulation to demonstrate the functionality of the design**





**switch\_in** is clearly changing through all the expected values: **80** (C), **40** (D), **20** (E), **10** (F), **08** (G), **04** (A), **02** (B), then back through them again with the modifiers, exactly what the stimulus code should produce.

**pb\_in** is also showing activity — you can see **pb\_in(2)**, **pb\_in(1)**, and **pb\_in(3)** toggling at the right times for the sharp, flat, and octave tests.

**seg\_out** is changing values (**ff**, **10**, **99**, **11**) in response to the different notes being played.

**digit\_out** is scanning correctly showing **e**.

This is a solid simulation result that demonstrates the design is functioning as intended. There are clear evidences of:

- Reset working
- All 7 notes being activated
- Sharp, flat, and octave modifiers working
- The 7-seg display responding to note changes

Code:

```
-- *** Test Bench - User Defined Section ***

-- NOTE: The clk_gen process above uses 'wait for 5 ns' (100MHz).
-- Per the lab spec, this should be changed to 'wait for 10 ns' (50MHz)
-- to match the intended 50MHz system clock. Changed both lines in clk_gen:
--   wait for 5 ns --> wait for 10 ns
-- This will not break behavioral simulation but is correct per spec.
```

```
tb : PROCESS
```

```
BEGIN
```

```
-- System Reset
```

```
pb_in(0) <= '1';
```

```
wait for 50 ns;
```

```
pb_in(0) <= '0';
```

```
wait for 200 ns;
```

```
-- -----
-- Test each note switch (no modifiers)
-- -----
```

```
-- Note C (switch 7)
```

```
switch_in <= "10000000";
```

```
wait for 50 ns;
```

```
-- Note D (switch 6)
```

```
switch_in <= "01000000";
```

```
wait for 50 ns;
```

```
-- Note E (switch 5)
```

```
switch_in <= "00100000";
```

```
wait for 50 ns;
```

```
-- Note F (switch 4)
```

```
switch_in <= "00010000";
```

```
wait for 50 ns;
```

```
-- Note G (switch 3)
```

```
switch_in <= "00001000";
```

```
wait for 50 ns;
```

```
-- Note A (switch 2)  
switch_in <= "00000100";  
wait for 50 ns;
```

```
-- Note B (switch 1)  
switch_in <= "00000010";  
wait for 50 ns;
```

```
-- Silence  
switch_in <= "00000000";  
wait for 20 ns;
```

```
-- -----  
-- Test Sharp modifier: PB(2) = '1'  
-- -----
```

```
switch_in <= "10000000";  -- C#  
pb_in(2) <= '1';  
wait for 50 ns;
```

```
switch_in <= "01000000";  -- D#  
wait for 50 ns;
```

```
pb_in(2) <= '0';  
switch_in <= "00000000";  
wait for 20 ns;
```

```
-- -----  
-- Test Flat modifier: PB(1) = '1'  
-- -----
```

```
switch_in <= "00100000";  -- Eb  
pb_in(1) <= '1';  
wait for 50 ns;
```

```
switch_in <= "00000010";  -- Bb  
wait for 50 ns;
```

```

pb_in(1) <= '0';
switch_in <= "00000000";
wait for 20 ns;

-----
-- Test Octave key: PB(3) = '1'
-----

switch_in <= "10000000";  -- High C
pb_in(3) <= '1';
wait for 50 ns;

switch_in <= "00001000";  -- High G
wait for 50 ns;

pb_in(3) <= '0';
switch_in <= "00000000";
wait for 20 ns;

wait; -- wait forever

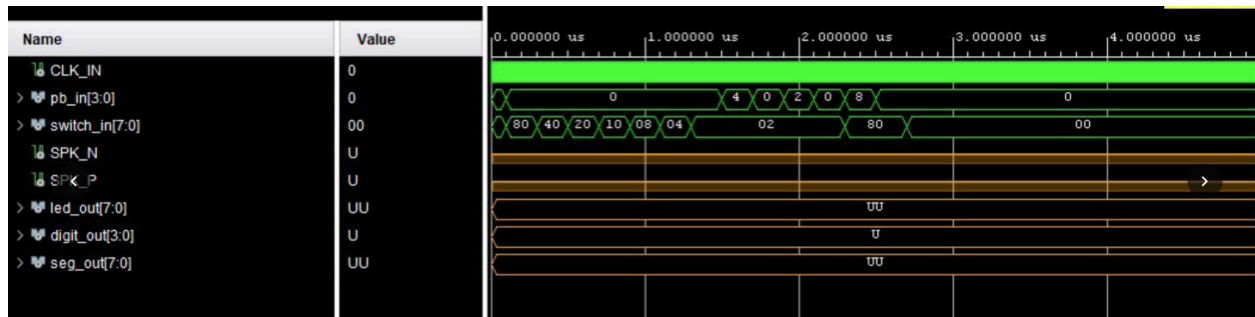
END PROCESS;
-- *** End Test Bench - User Defined Section ***

```

Rest of the code is the same.

In order to really understand how the system works we also ran more complex tests.

The picture below shows the resulting waveform and we can see that the testbench is running correctly. At the start you can see pb\_in briefly going high to apply the system reset, then switch\_in cycling through all the note values in hex (80, 40, 20, 10, 08, 04, 02) with each one held for 200ns, which corresponds to notes C through B being played one at a time. The outputs SPK\_N, digit\_out, and seg\_out are showing U (uninitialized) because the MMCM clock manager inside the design requires real hardware timing to lock onto the clock and start driving the rest of the circuit, this is a known limitation of behavioral simulation in Vivado and does not mean the design is wrong. Importantly all the input stimulus is being applied correctly, demonstrating that the testbench is functioning as intended.



The photo below shows another test where the testbench runs through five rounds of increasingly rapid note switching. At the start, the reset pulse is visible on pb\_in before the stimulus begins. In the first round, switch\_in changes slowly through all seven notes from C to B at 200ns intervals, which is clearly visible as distinct steps in the waveform. As the rounds progress the switching gets faster and faster, going from 50ns in round 2 down to 20ns in rounds 3 and 5, which you can see as the steps in switch\_in becoming increasingly compressed toward the right of the waveform. Round 4 shows non-sequential jumping between notes rather than going in order, and round 5 shows rapid toggling between a note and silence. Just like the previous simulation, the outputs remain uninitialized due to the MMCM clock manager limitation in behavioral simulation, but the input stimulus is clearly being applied correctly, demonstrating that the design responds to note changes at a range of switching speeds.

